8Byte DevOps Assignment Challenges and Resolutions

I used Spring PetClinic to focus on the infrastructure, deployment and monitoring work. This document covers the main issues faced during the assignment, their causes, the resolutions and the checks made afterwards. Shared-host isolation and recovery testing still have limits, which are explained below.

Infrastructure setup

1 EC2 bootstrap failed

Problem Terraform created EC2, but the first-boot script did not complete. The script requested awscli2, which was not the correct package name in the Amazon Linux 2023 repository. Docker setup therefore did not finish.

Resolution The package name was corrected to awscli-2. The replacement plan was reviewed before applying it because a change to this bootstrap configuration replaces the instance.

Verification The replacement host completed cloud-init. Docker and SSH were active, an outbound HTTPS request returned 200, and a Docker Hub test image was pulled and run successfully. This showed why instance creation alone was not enough to confirm readiness.

2 GitHub OIDC trust did not match

Problem GitHub Actions could not assume the deployment role. The trust policy expected a repository name-based subject, while the repository used an immutable subject containing the owner and repository IDs.

Resolution The actual subject configuration was checked through the GitHub API. Both role trust policies were updated to match it. Deployment stayed restricted to main, while the notification role allowed main and pull-request subjects.

Verification The updated trust policies were confirmed in AWS. A later staging workflow obtained temporary credentials and built, scanned and pushed the image. No long-lived AWS access keys were added to GitHub.

3 Deleted secret names were still reserved

Problem After cleanup, application secrets remained scheduled for deletion. Their names could not be reused immediately for a fresh deployment.

Resolution A stable database_secret_suffix was added for each new deployment. The deployment scripts and both release workflows were updated to use the same suffix. The initializer and EC2 policy use the generated secret ARNs.

Verification Later AWS inventories showed secrets with the new suffixes. The release log also passed DATABASE_SECRET_SUFFIX to EC2. The value must stay unchanged during a deployment and match the Terraform input.

SSH access and database setup

4 Deployment targeted an old EC2 instance

Problem SendSSHPublicKey returned AccessDenied when the workflow targeted an instance from the previous deployment. The role is scoped to the managed application instance and the ec2-user account.

Resolution The required correction was to align EC2_INSTANCE_ID with the current Terraform output and check the applied role policy. Broadening access to every instance would have removed the intended restriction.

Verification The next deployment log targeted the replacement instance and reached SSH host-key checking. A later log ran commands on EC2, confirming that the public-key permission check and SSH connection had passed.

5 SSH rejected the replacement host key

Problem The staging workflow stopped with REMOTE HOST IDENTIFICATION HAS CHANGED. GitHub still had the previous server public key saved in EC2_SSH_HOST_KEY, while the replacement EC2 presented a different key.

Resolution The new host public key and fingerprint needed verification through the EC2 connection. The GitHub variable was then aligned with that key, keeping strict SSH checking enabled. The public host key is different from the temporary login key.

Verification The next workflow log contained the new public key and reached the image pull on EC2. Host-key checking and SSH had therefore passed. An instance replacement requires this variable to be refreshed.

6 The database secret had no current value

Problem Deployment reached EC2 but GetSecretValue failed because Secrets Manager had no value labelled AWSCURRENT. Terraform had created the secret container, but the application credentials were not ready.

Resolution The setup order was documented as infrastructure apply, database initialization, then deployment. The operator command is python3 8Byte-Infra/scripts/initialize_databases.py --execute. It populates the application secrets and creates the database users.

Verification Earlier initialization checks confirmed populated secrets, successful own-database logins, cross-database denial and persistence. The later empty-secret failure needs the same initialization step after recreation; a successful rerun of that specific failed job was not recorded.

Releases and environment separation

7 Production received the wrong workflow run ID

Problem Production promotion was started with a production workflow run ID. The validation rejected it because staging_run_id must identify a successful staging release from a push to main.

Resolution The correct procedure is to open a successful Build and deploy to staging run and copy the number after /actions/runs/ in its URL. The production workflow checks the workflow path, event, branch, completion and staging deployment result.

Verification The failure log correctly identified a production workflow_dispatch run and rejected it before deployment. An earlier successful promotion also recorded that staging and production ran the same image digest.

8 Shared resources needed clear isolation limits

Problem Staging and production shared EC2 and RDS to keep the assignment manageable. Separate Docker networks and database names alone did not prove that one environment could not reach the other.

Resolution Separate database users were created, PUBLIC and cross-database access were revoked, and each user owned only its database. Containers used separate names, networks, ports and resource limits. Connection and persistence checks were added.

Verification Database cross-access was denied and environment data stayed separate. However, earlier testing could reach production through the shared host published port from staging. Network isolation was therefore incomplete despite database separation.

Cleanup and remaining checks

9 Terraform destroy stopped on protected resources

Problem An intentional destroy stopped because the ECR repository still contained images and RDS deletion protection was enabled. Some resources had already been removed, leaving a partial cleanup.

Resolution The remaining resources were refreshed and reviewed. ECR images were explicitly removed, and database deletion protection was disabled for the approved cleanup. A fresh destroy plan was used to finish the operation, with a final database snapshot preserved at that stage.

Verification The recorded cleanup apply destroyed the eight remaining managed resources. Terraform state was empty and a fresh destroy plan showed no changes. AWS checks confirmed the project compute, network, database and pipeline resources were gone.

10 Storage needed a separate final cleanup

Problem The manually created state bucket and retained database snapshot were outside normal stack deletion. The ALB log bucket and state bucket also contained object versions, so current-object removal alone would not empty them.

Resolution After confirming infrastructure cleanup, all versions and deletion markers were removed from the two named project buckets. Both buckets and the retained final RDS snapshot were then explicitly deleted. The state bucket was removed only after its state was confirmed empty.